

```
1 !pip install librosa soundfile
```

```
Requirement already satisfied: librosa in /usr/local/lib/python3.12/dist-packages (0.11.0)
Requirement already satisfied: soundfile in /usr/local/lib/python3.12/dist-packages (0.13.1)
Requirement already satisfied: audioread>=2.1.9 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: numba>=0.51.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: numpy>=1.22.3 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: scikit-learn>=1.1.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: joblib>=1.0 in /usr/local/lib/python3.12/dist-packages (from l
Requirement already satisfied: decorator>=4.3.0 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: pooch>=1.1 in /usr/local/lib/python3.12/dist-packages (from li
Requirement already satisfied: soxr>=0.3.2 in /usr/local/lib/python3.12/dist-packages (from l
Requirement already satisfied: typing_extensions>=4.1.1 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: lazy_loader>=0.1 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: msgpack>=1.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: cffi>=1.0 in /usr/local/lib/python3.12/dist-packages (from sou
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages (from cff
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from laz
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: platformdirs>=2.5.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages
```

```
1 import zipfile
2 import os
3
4 zip_file_path = '/content/wav.zip'
5 extraction_path = '/content/wav2'
6
7 print(f"Unzipping {zip_file_path} to {extraction_path}...")
8 with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
9     zip_ref.extractall(extraction_path)
10 print("Unzipping complete.")
11
12 print("\nContents of extracted directory:")
13 for root, dirs, files in os.walk(extraction_path):
14     level = root.replace(extraction_path, '').count(os.sep)
15     indent = ' ' * 4 * (level)
16     print(f'{indent}{os.path.basename(root)}/')
17     subindent = ' ' * 4 * (level + 1)
18     for f in files[:5]:
19         print(f'{subindent}{f}')
20     if len(files) > 5:
21         print(f'{subindent}...')
22     break
```

```
Unzipping /content/wav.zip to /content/wav2...
Unzipping complete.
```

```
Contents of extracted directory:
wav2/
```

```

1 import os
2 import librosa
3 import numpy as np
4 from sklearn.model_selection import train_test_split
5 from sklearn.svm import SVC
6 from sklearn.metrics import accuracy_score, classification_report
7 from sklearn.preprocessing import LabelEncoder
8 import pickle
9 import warnings
10 warnings.filterwarnings('ignore')
11
12 print("Libraries successfully imported!")

```

Libraries successfully imported!

```

1 import os
2 import librosa
3 import pandas as pd
4 import numpy as np
5
6 def perform_audio_eda(dataset_path):
7     data_list = []
8     print(f"Analyzing dataset at: {dataset_path}...")
9
10    for speaker in os.listdir(dataset_path):
11        speaker_path = os.path.join(dataset_path, speaker)
12        if os.path.isdir(speaker_path):
13            files = [f for f in os.listdir(speaker_path) if f.endswith('.wav')]
14            for f in files:
15                file_path = os.path.join(speaker_path, f)
16                duration = librosa.get_duration(path=file_path)
17                data_list.append({
18                    'speaker': speaker,
19                    'file_name': f,
20                    'duration_sec': duration
21                })
22
23    df = pd.DataFrame(data_list)
24
25    print("\n--- Dataset Summary ---")
26    print(f"Total Speakers: {df['speaker'].nunique()}")
27    print(f"Total Audio Files: {len(df)}")
28    print(f"Average Duration: {df['duration_sec'].mean():.2f} seconds")
29    print(f"Shortest Clip: {df['duration_sec'].min():.2f} seconds")
30    print(f"Longest Clip: {df['duration_sec'].max():.2f} seconds")
31
32    print("\n--- Speaker Breakdown ---")
33    breakdown = df.groupby('speaker')['duration_sec'].agg(['count', 'mean', 'sum']).reindex(sorted(df['speaker'].unique()), fill_value=0)
34    print(breakdown)
35
36    return df
37
38 eda_df = perform_audio_eda("/content/wav2/wav")

```

Analyzing dataset at: /content/wav2/wav...

```

--- Dataset Summary ---
Total Speakers: 4

```

Total Audio Files: 377
 Average Duration: 7.04 seconds
 Shortest Clip: 3.96 seconds
 Longest Clip: 42.76 seconds

--- Speaker Breakdown ---

	File Count	Avg Duration	Total Duration
speaker			
Eartha_Kitt	158	6.612974	1044.849875
Ed_Westwick	73	6.011569	438.844562
Eddie_Griffin	50	6.741663	337.083125
Eddie_Izzard	96	8.672146	832.526000

```

1 file_path="/content/wav2/wav"
2 def extract_features(file_path):
3     try:
4         audio, sample_rate = librosa.load(file_path, sr=16000)
5         mfccs = librosa.feature.mfcc(y=audio, sr=sample_rate, n_mfcc=40)
6         mfccs_processed = np.mean(mfccs.T, axis=0)
7         return mfccs_processed
8     except Exception as e:
9         print(f"Error processing {file_path}: {e}")
10        return None
11
12 print("Feature extraction function ready!")

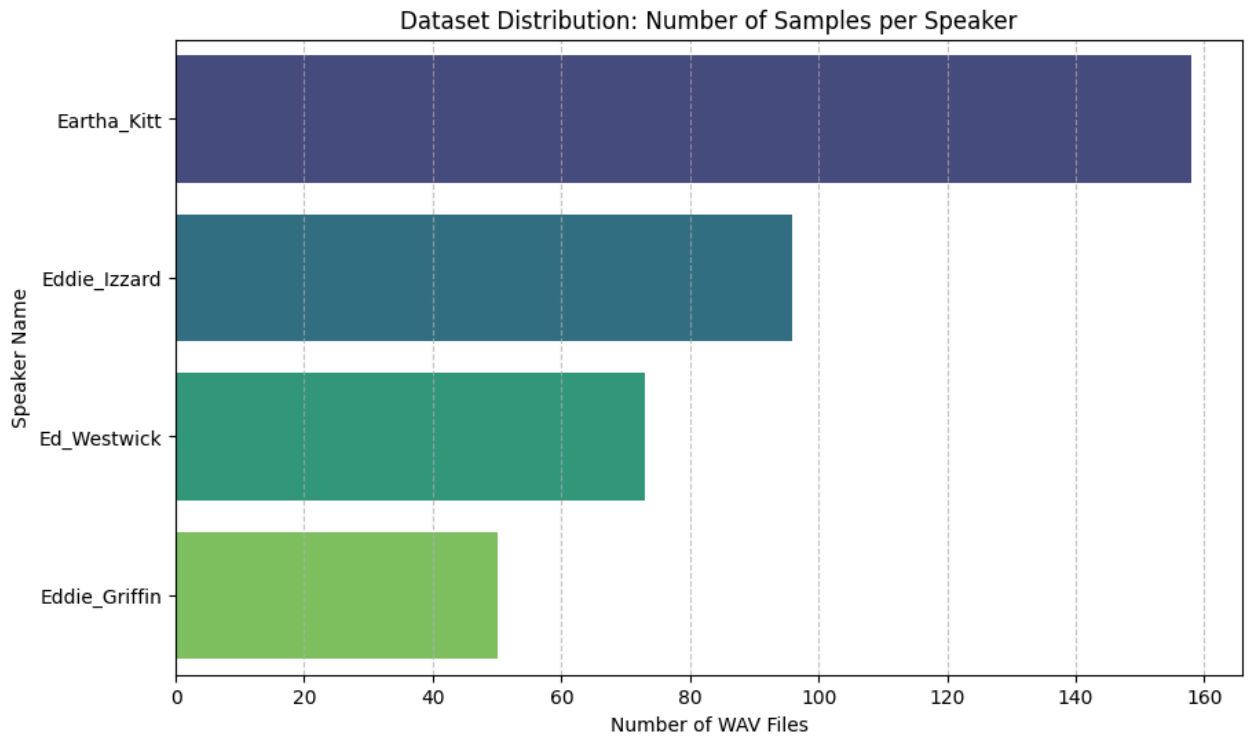
```

Feature extraction function ready!

```

1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 import pandas as pd
4
5 unique, counts = np.unique(y, return_counts=True)
6 speaker_counts = dict(zip(unique, counts))
7
8 df_stats = pd.DataFrame(list(speaker_counts.items()), columns=['Speaker', 'Sample Count'])
9 df_stats = df_stats.sort_values(by='Sample Count', ascending=False)
10
11 plt.figure(figsize=(10, 6))
12 sns.barplot(x='Sample Count', y='Speaker', data=df_stats, palette='viridis')
13 plt.title('Dataset Distribution: Number of Samples per Speaker')
14 plt.xlabel('Number of WAV Files')
15 plt.ylabel('Speaker Name')
16 plt.grid(axis='x', linestyle='--', alpha=0.7)
17 plt.show()

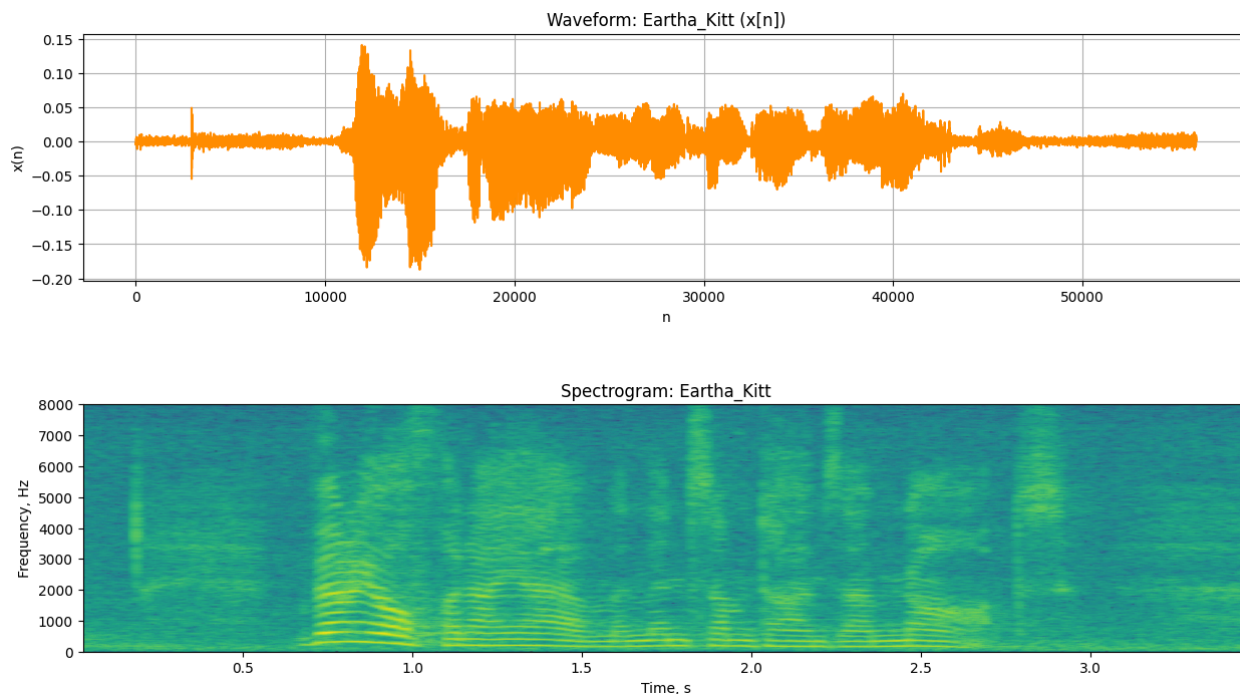
```



```

1 import matplotlib.pyplot as plt
2 import librosa
3 import numpy as np
4
5 # Load a file from a different speaker: Eartha_Kitt
6 sample_rate = 16000
7 test_file = "/content/wav2/wav/Eartha_Kitt/5r0dWxy17C8_00001.wav"
8 signal, sr = librosa.load(test_file, sr=sample_rate)
9
10 # Keep the first 3.5 seconds
11 signal = signal[0:int(3.5 * sample_rate)]
12
13 plt.figure(1, figsize=(15, 8))
14
15 # Waveform Plot
16 plot_a = plt.subplot(211)
17 plt.subplots_adjust(wspace=0, hspace=0.5)
18 plot_a.plot(signal, color='darkorange')
19 plot_a.set_title('Waveform: Eartha_Kitt (x[n])')
20 plot_a.set_xlabel('n')
21 plot_a.set_ylabel('x(n)')
22 plot_a.grid(True)
23
24 # Spectrogram Plot
25 plot_b = plt.subplot(212)
26 plot_b.specgram(signal, NFFT=1024, Fs=sample_rate, noverlap=900, cmap='viridis')
27 plot_b.set_title('Spectrogram: Eartha_Kitt')
28 plot_b.set_xlabel('Time, s')
29 plot_b.set_ylabel('Frequency, Hz')
30
31 plt.show()

```



```

1 def load_data(dataset_path):
2     X = []
3     y = []
4
5     for person_name in os.listdir(dataset_path):
6         person_folder = os.path.join(dataset_path, person_name)
7
8         if not os.path.isdir(person_folder):
9             continue
10
11        for file_name in os.listdir(person_folder):
12            if file_name.endswith('.wav'):
13                file_path = os.path.join(person_folder, file_name)
14                features = extract_features(file_path)
15
16                if features is not None:
17                    X.append(features)
18                    y.append(person_name)
19
20    return np.array(X), np.array(y)
21
22 dataset_directory = "/content/wav2/wav"
23 X, y = load_data(dataset_directory)
24
25 print("Dataset loading function ready!")

```

Dataset loading function ready!

```

1 label_encoder = LabelEncoder()
2 y_encoded = label_encoder.fit_transform(y)
3
4 X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.2, random
5

```

```

6 print("Model training start ho rahi hai...")
7 model = SVC(kernel='linear', probability=True)
8 model.fit(X_train, y_train)
9
10 predictions = model.predict(X_test)
11 accuracy = accuracy_score(y_test, predictions)
12
13 print(f"\nModel Training Complete! Accuracy: {accuracy * 100:.2f}%")
14 print("\nClassification Report:\n", classification_report(y_test, predictions, target_r
15
16 with open('speaker_model.pkl', 'wb') as f:
17     pickle.dump(model, f)
18 with open('label_encoder.pkl', 'wb') as f:

```

Model training start ho rahi hai...

\nModel Training Complete! Accuracy: 97.37%

\nClassification Report:\n

			precision	recall	f1-score	support
Eartha_Kitt	1.00	1.00	1.00	23		
Ed_Westwick	0.88	1.00	0.94	15		
Eddie_Griffin	1.00	0.91	0.95	11		
Eddie_Izzard	1.00	0.96	0.98	27		
accuracy			0.97	76		
macro avg	0.97	0.97	0.97	76		
weighted avg	0.98	0.97	0.97	76		

```

1 import librosa
2 import soundfile as sf
3 import os
4
5 def recognize_speaker(audio_path, trained_model, encoder):
6     features = extract_features(audio_path)
7
8     if features is not None:
9         features = features.reshape(1, -1)
10        predicted_id = trained_model.predict(features)[0]
11        person_name = encoder.inverse_transform([predicted_id])[0]
12
13        probabilities = trained_model.predict_proba(features)[0]
14        confidence = probabilities[predicted_id] * 100
15
16        return person_name, confidence
17    else:
18        return "Audio process nahi ho saka", 0
19
20 test_audio_file = "/content/x6uYqmx31kE_00019.wav"
21 predicted_name, conf = recognize_speaker(test_audio_file, model, label_encoder)
22 print(f"Prediction: Ye awaz {predicted_name} ki lag rhi hai (Confidence: {conf:.2f}%")

```

Prediction: Ye awaz Eartha_Kitt ki lag rhi hai (Confidence: 93.44%)

